COMP 305 Full Stack Tutorial

Kai Laitinen

Setup and Install

You will need:

- Microsoft SQL Server Management Studio database
- An Express Microsoft SQL Server
- A GitHub account
- Microsoft Visual Studio
- Node.js
- Microsoft Visual Studio Code
- GitBash

All of which should be up to the latest versions, you can check to make sure they are in your installer(s). Microsoft Visual Studio should also have the ASP.NET packages installed with the initial installation.

SSMS: Install SQL Server Management Studio | Microsoft Learn
SSMS Server: SQL Server Downloads | Microsoft
MVS: Visual Studio: IDE and Code Editor for Software Development
Node.js: Node.js — Download Node.js®
VSC: Visual Studio Code - The open source AI code editor | Your home for multi-agent development
GitBash: Git - Install for Windows

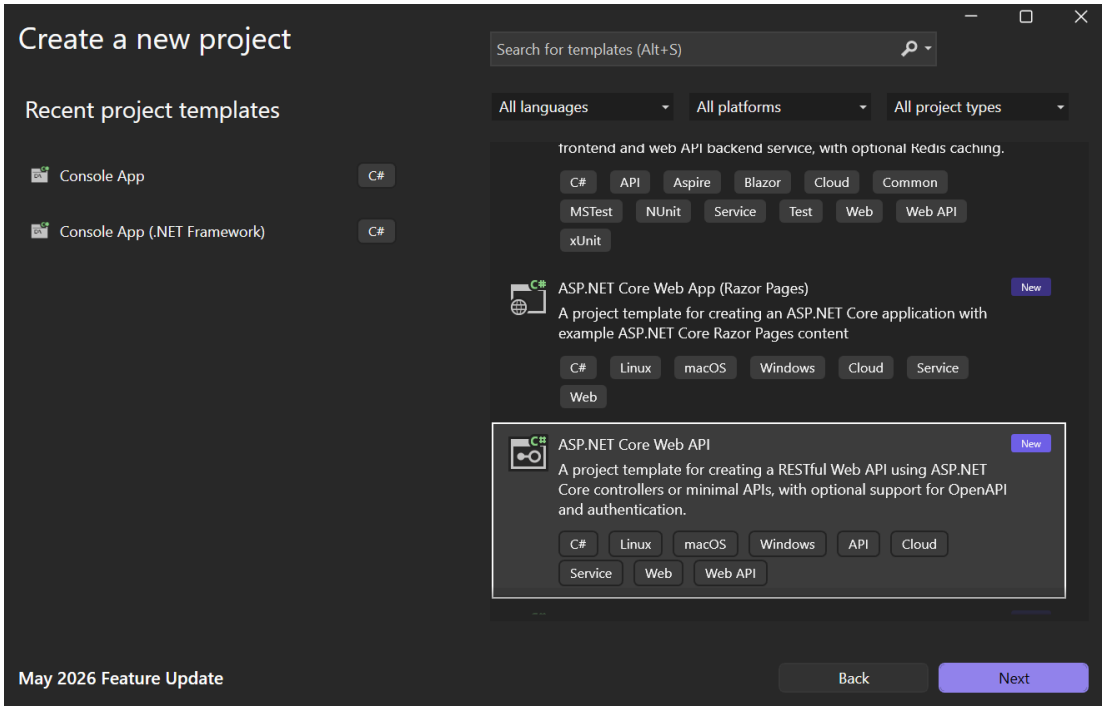
If you have trouble downloading SSMS, a server, connection with the server (check Trust Server Certificate) and creating tables, look it up or use AI. This is a tutorial for the front end and back end basics.

Starting In Microsoft Visual Studio

IF YOU DID NOT INSTALL ASP.NET with your Microsoft Visual Studio, go back to your installer and do so.

In SSMS, make sure your table(s) has a primary key with identity (1,1) for the models we will be scaffolding.

We will be using this template for your project, if you cannot see it in your templates, scroll down, search and then check if you installed ASP.NET with your MVS.



Create .NET Web API basic solution.

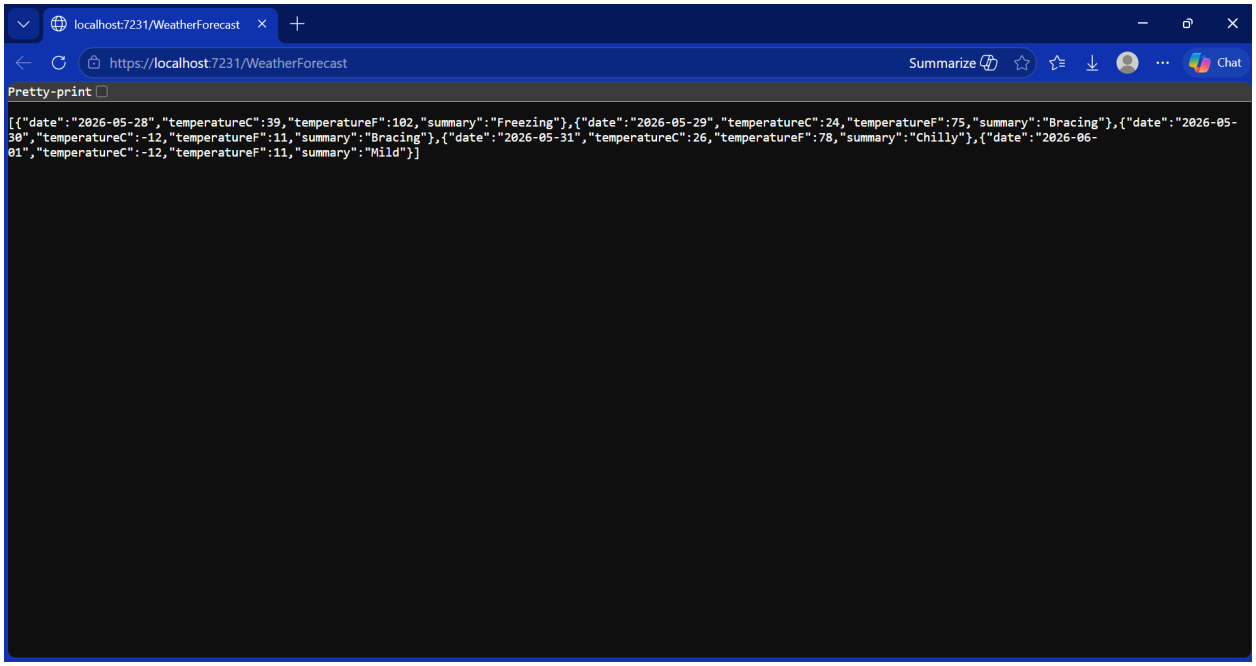
Back End

Basic back end with API endpoints in MVS

Open your Solution Explorer from View, it is vital for everything we will be doing.

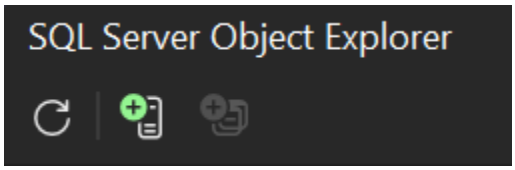
Initial Test

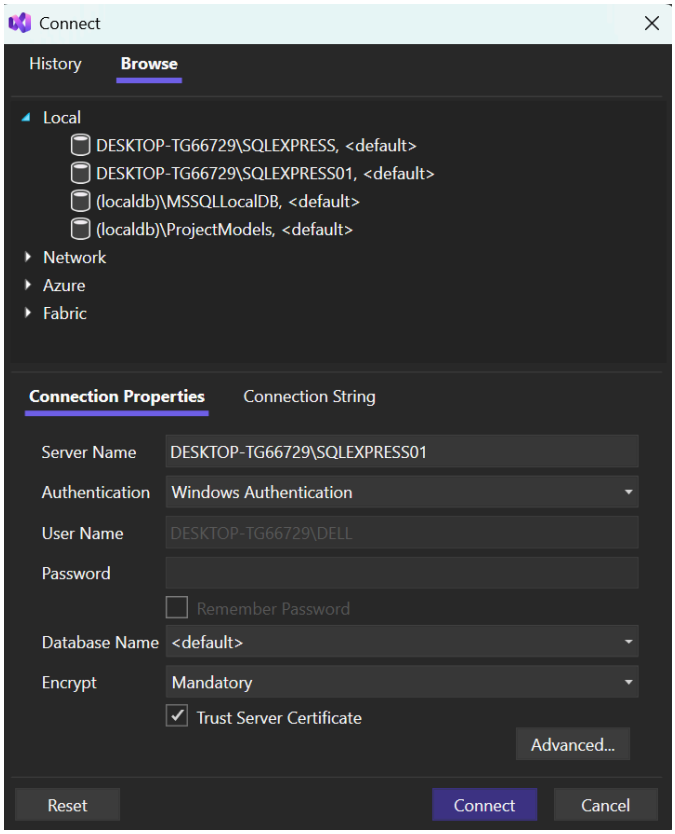
Go to Properties and open launchsettings.json.
Change either the https or http profile to have *“launchBrowser”: true* and add *“launchUrl”: “WeatherForecast”*. If there is a problem in launching this, other than trusting localhost and Hot Reload, figure it out yourself, this is a learning opportunity. Otherwise, you should get this:



SQL Server

To connect a server, go to View=>SQL Server Object Explorer=>Add DB=>Browse=>Local=>(YOUR DB).





NuGet

Go to Project=>Manage NuGet Packages…=>Browse

Download STD Microsoft.EntityFrameworkCore packages. (.SqlServer, Tools, and Design)

Install the latest versions of each.

Put this string with your connection string into the Git package terminal in Tools=>NuGet Package Manager=>Package Manager Console:

Scaffold-DbContext “ConnectionString” Microsoft.EntityFrameworkCore.SqlServer

Your Connection string is in the SQL Server Object Explorer inside the properties of the server instance. But, you must add *“Initial Catalog=DATABASENAME;”* inside the connection string to specify your database inside the server.

This makes your models.

Connection String Setup

In appsettings.json add

“ConnectionStrings” : { “DefaultConnection” : “YourConnectionString”}

Make sure this is not inside *“logging”* of appsettings.json

In Program.cs add (before var app = builder.Build();)

builder.Services.AddDbContext<YOUR_DB_CONTEXT_HERE>(options => options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

And

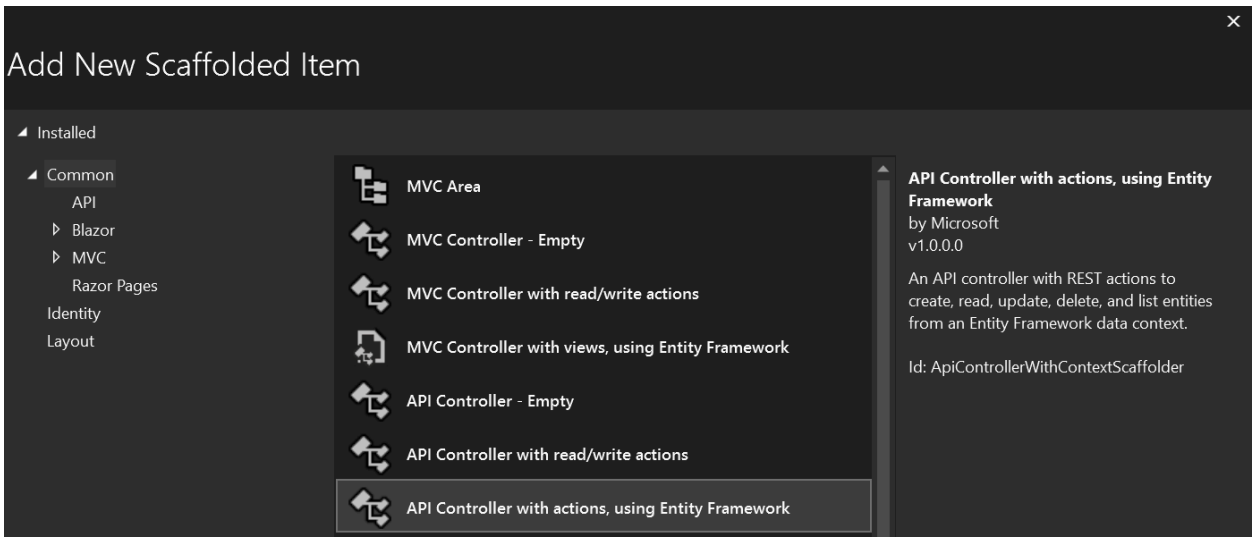
using Microsoft.EntityFrameworkCore;
using <Solution>;

At the top, if they did not already appear.

RUN THE SOLUTION TO MAKE SURE IT WAS NOT BROKEN IN THE PROCESS

Controllers

We will now make a controller, starting with Controllers=>Add=>New Scaffolded Item=>API Controller with actions, using Entity Framework



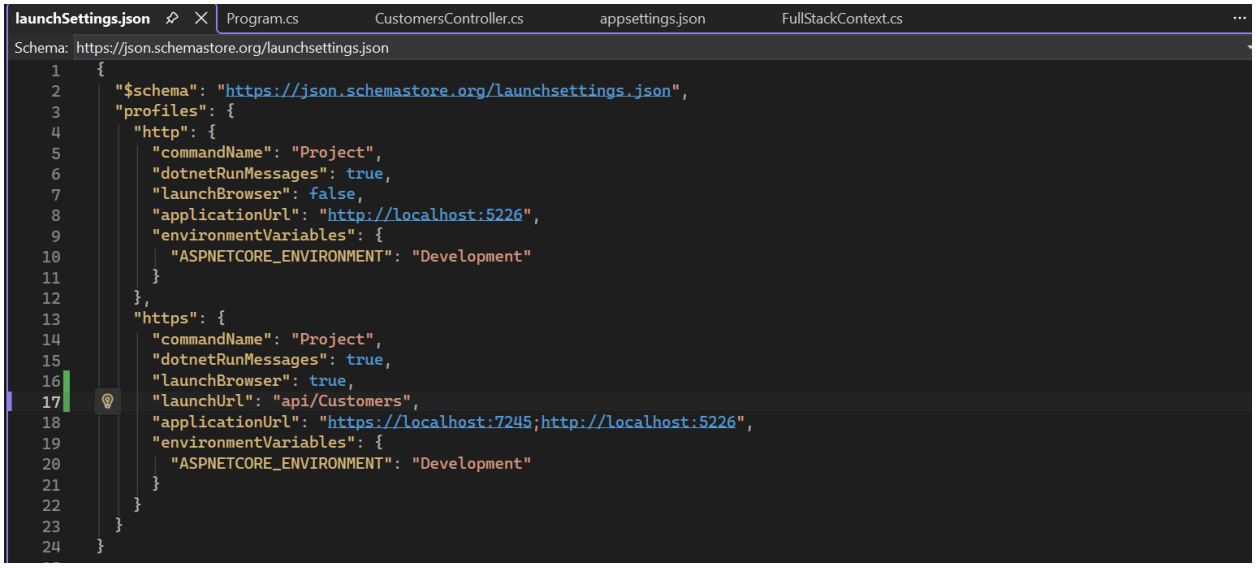
This should build out a controller for a table of your given scaffolded database.

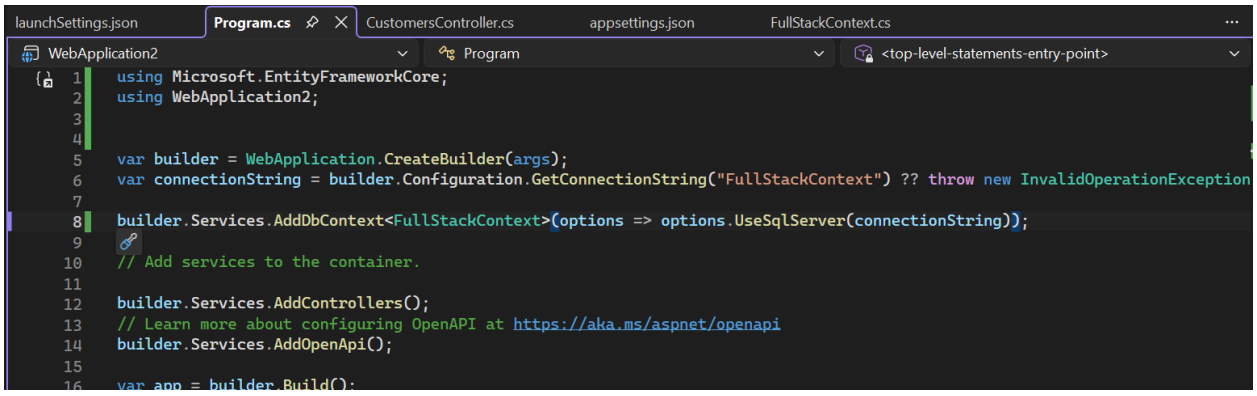
We can now change the launchUrl to *“api/Customer”* (in this example) and run the solution to get this simple back end.

This should set everything up for you. Then go to launchSettings in Properties and change the launchUrl to *“api/<table + s>”*

Then go into program.cs and add *using <NameOfSolution>*

Running this should make a window appear with JSON data. This is how mine looks:





launchSettings.json Program.cs **CustomersController.cs** appsettings.json FullStackContext.cs

WebApplication2 | CustomersController | DeleteCustomer(long? id)

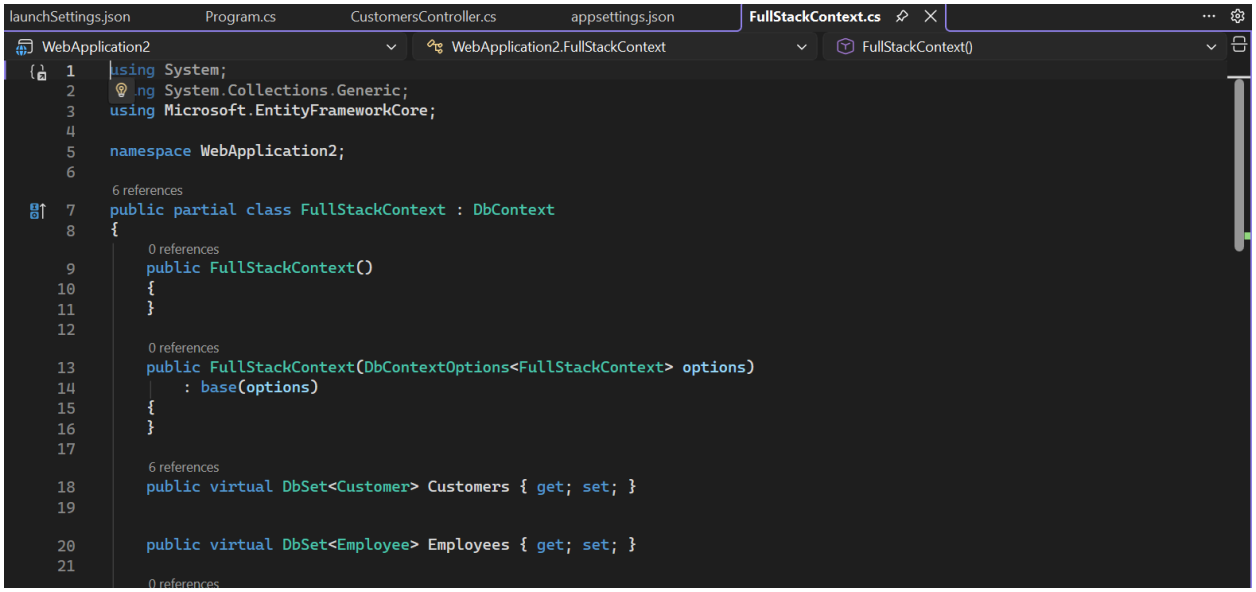
```
1   using Microsoft.AspNetCore.Mvc;
2   using Microsoft.EntityFrameworkCore;
3   using WebApplication2;
4
5   [Route("api/[controller]")]
6   [ApiController]
    1 reference
7   public class CustomersController : ControllerBase
8   {
9       private readonly FullStackContext _context;
        0 references
10      public CustomersController(FullStackContext context)
11      {
12          _context = context;
13      }
14
15      // GET: api/Customer
16      [HttpGet]
        0 references
17      public async Task<ActionResult<IEnumerable<Customer>>> GetCustomer()
18      {
19          return await _context.Customers.ToListAsync();
20      }
21
22      // GET: api/Customer/5
23      [HttpGet("{id}")]

24      public async Task<ActionResult<Customer>> GetCustomer(long id)
25      {
```

launchSettings.json Program.cs CustomersController.cs **appsettings.json** FullStackContext.cs

Schema: https://www.schemastore.org/appsettings.json

```
1
2   'Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   'AllowedHosts": "*",
9   'ConnectionStrings": {
10    "DefaultConnection": "Data Source=DESKTOP-TG66729\\SQLEXPRESS01;Initial Catalog=FullStack;Integrated Security=True;Connect T
11    "FullStackContext": "Server=(localdb)\\mssqllocaldb;Database=FullStackContext;Trusted_Connection=True;MultipleActiveResultSe
12  }
13
```



Front End

Basic back end in Visual Studio Code using Node.js

At this point it is essential to have everything from the Setup and Install section installed, as this is where it will all work together.
First we will make sure Node is installed and in VSC, go to the terminal and

node -v

npm -v

If there is an error, that is expected and next we will put

Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned

This gives us the ability to run commands in VSC so we can use

npx create-react-app my-frontend

This will create a react front end app with the name my-frontend (or whatever name you would like, I like app1)

Now use

cd my-frontend

code .

To Isolate VSC to just out frontend

Open src=>app.js and we will overhaul the app to make it our own

App.js

src > App.js > default

```jsx
import { useEffect, useState } from "react";

function App() {
  const [customers, setCustomers] = useState([]);

  useEffect(() => {
    fetch("https://localhost:7245/api/Customers")
      .then(res => res.json())
      .then(data => setCustomers(data))
      .catch(err => console.error(err));
  }, []);

  return (
    <div>
      <h1>Customers</h1>
      <ul>
        {customers.map(c => (
          <li key={c.id}>{c.name} – {c.email}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Now we return to MVS to add a way to connect to the backend via

EmployeesController.cs launchSettings.json Program.cs CustomersController.cs appsettings.json FullStackContext.cs

WebApplication2 Program <top-level-statements-entry-point>

```
using Microsoft.EntityFrameworkCore;
using WebApplication2;


var builder = WebApplication.CreateBuilder(args);
var                                    on.GetConnectionString("FullStackContext") ?? throw new InvalidOperationException

bui                                    text>(options => options.UseSqlServer(connectionString));

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact", policy =>
        policy.WithOrigins("http://localhost:3000")
              .AllowAnyHeader()
              .AllowAnyMethod());
});


var app = builder.Build();
```

(local variable) WebApplicationBuilder? builder

'builder' is not null here.

Describe

This must come before the app builder.

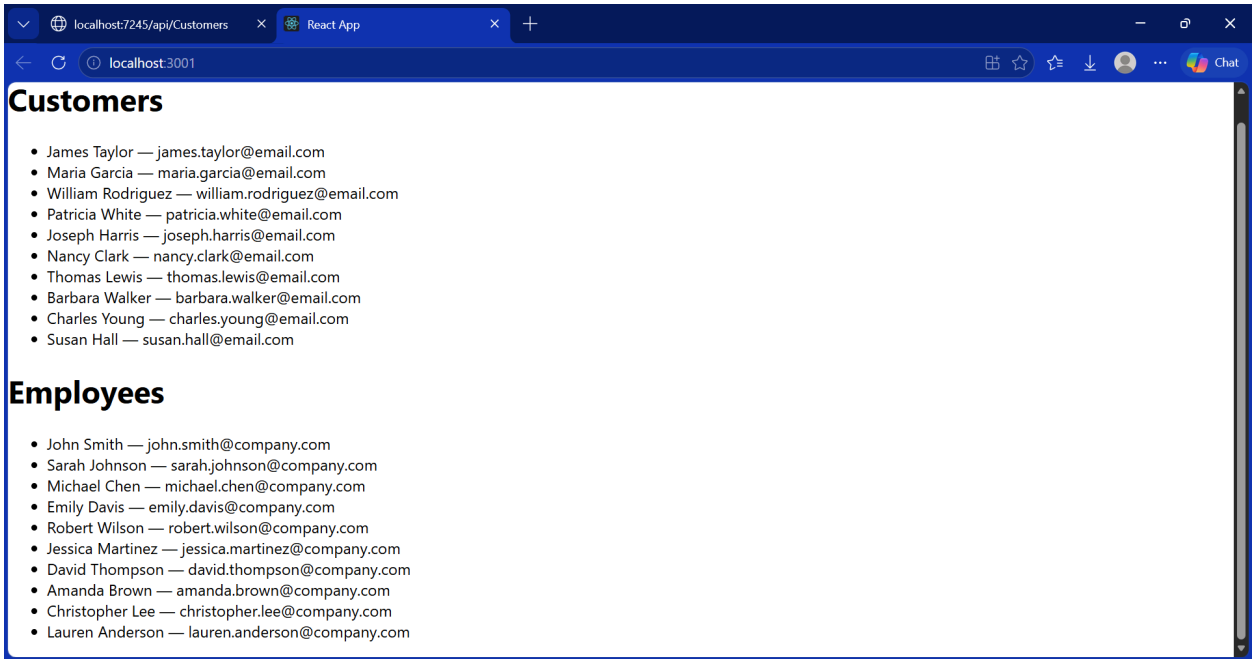
Then we add

```
27  // Configure the HTTP request pipeline.
28  if (app.Environment.IsDevelopment())
29  {
30      app.MapOpenApi();
31  }
32
33  app.UseCors("AllowReact");
34
35  app.UseHttpsRedirection();
36
37  app.UseAuthorization();
38
39  app.MapControllers();
40
41  app.Run();
```

Before app.Run()

This creates a Cors policy to allow our specific frontend and then adds it to the backend app's functions so the frontend is allowed to access it.

We can now run the backend with the run button is MVS and use npm start is VSC to get a quaint frontend of our database:



We now have a basic read function.

READ

Again, start the front end with
const getStudents= () => {

```
        fetch("https://localhost:####/api/Students)
        .then(res => res.json())
        .then(setApiData)
        .catch(err => setError(err.message))
}

useEffect(() => { getStudents() }, [])
```

We now add a apiData.map for the App so we can see the Students:

```
return (
    <div>
        <h1>Students</h1>
        {apiData?.map((s: any) => (
            <div key={s.studentId}> <span style={{ textAlign: "left", fontWeight: "bold" }}>{s.name}</span>
                <p>{s.studentId}, {s.name}, {s.email} </p>

            </div>
        ))}
```

This sets us up to be able to access the data and also access the data for other CRUD functions.
This will let us have a way to check our database table, as well as recheck it after we change it with the other CRUD functions.

CREATE

For our first alternate CRUD function, create will be an example of how the front end uses the back end's functions to change the database data.
The formula for these goes like this: fetch the basic backend, specify the method we are using, use the corresponding headers, then add any additional data into the body.
Like this:

```
const postStudent = () => {
        fetch(FETCH, {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ name: newName, email: newEmail }) })
                .then(() => { setNewName(""); setNewEmail(""); getStudents() })
                .catch(err => setError(err.message))
}
```

This effectively passes the variables for the POST back end function.

Now we must add it to our front end App. For this we will add input sections and then a button to execute the post function.
Here is what we have added for postStudent.

```
const postStudent = () => {
    fetch(FETCH, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ name: newName, email: newEmail })
    })
        .then(() => { setNewName(""); setNewEmail(""); getStudents() })
        .catch(err => setError(err.message))
}


if (loading) return <p>Loading...</p>
if (error) return <p style={{ color: "red" }}>Error: {error}</p>

return (
    <div>
        <h1>Students</h1>
        {apiData?.map((s: any) => (
            <div key={s.studentId}> <span style={{ textAlign: "left", fontWeight: "bold" }}>{s.name}</span>
                <p>{s.studentId}, {s.name}, {s.email} </p>
                <input value={newName} onChange={e => setNewName(e.target.value)} placeholder="Name" />
                <input value={newEmail} onChange={e => setNewEmail(e.target.value)} placeholder="Email" />
                <button onClick={postStudent}>Add Student</button>
            </div>
        ))}
    </div>
```

Correction: the inputs should be outside the second level of so it shows up after instead of on each one.

UPDATE

This uses the same formula as the create but it requires an id input as well.

const putStudent = (id: number) => {
fetch(`${FETCH}/${id}`, {
method: "PUT",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ name: editName, email: editEmail })
})
.then(() => { setEditId(null); setEditName(""); setEditEmail("");
getStudents()})
.catch(err => setError(err.message))
}

```
52      const putStudent = (id: number) => {
53          fetch(`${FETCH}/${id}`, {
54              method: "PUT",
55              headers: { "Content-Type": "application/json" },
56              body: JSON.stringify({ StudentId: editId, name: editName, email: editEmail })
57          })
58              .then(() => { setEditId(""); setEditName(""); setEditEmail(""); getStudents() })
59              .catch(err => setError(err.message))
60      }
```

```
85              <input value={editId} onChange={e => setEditId(e.target.value)} placeholder="Edit Id" />
86              <input value={editName} onChange={e => setEditName(e.target.value)} placeholder="Edit Name" />
87              <input value={editEmail} onChange={e => setEditEmail(e.target.value)} placeholder="Edit Email" />
88              <button onClick={() => putStudent(Number(editId))}>Update Student</button>
```

Very key to match the names of the model in MVS to the variables you pass here, or else the update will not work.

DELETE

```
const deleteStudent = (id: number) => {
    fetch(`${FETCH}/${id}`, {
      method: "DELETE"
    })
      .then(() => getStudents())
      .catch(err => setError(err.message))
  }
```

```
89              <input value={deleteId} onChange={e => setDeleteId(e.target.value)} placeholder="Delete Id" />
90              <button onClick={() => deleteStudent(Number(deleteId))}>Delete Student</button>
91          </div>
```

Final App

main.tsx | App4.tsx 2 | App.tsx

src > App4.tsx > App

```tsx
import { useEffect, useState } from "react";

function App() {
    const [apiData, setApiData] = useState<any>(null)
    const [error, setError] = useState<string | null>(null)
    const [loading, setLoading] = useState(true)

    const FETCH = "https://localhost:7075/api/Student1"

    const [newName, setNewName] = useState("")
    const [newEmail, setNewEmail] = useState("")
    const [editName, setEditName] = useState("")
    const [editEmail, setEditEmail] = useState("")
    const [deleteId, setDeleteId] = useState("")

    const getStudents = () => {
        fetch(FETCH)
            .then(res => res.json())
            .then(setApiData)
            .catch(err => setError(err.message))
    }

    useEffect(() => { getStudents() }, [])


    useEffect(() => {
        fetch(FETCH)
            .then(res => res.json())
            .then(setApiData)
            .catch(() => setError("Failed to connect to Database"))
            .finally(() => setLoading(false))
    }, [])

    const postStudent = () => {
        fetch(FETCH, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ name: newName, email: newEmail })
        })
            .then(() => { setNewName(""); setNewEmail(""); getStudents() })
            .catch(err => setError(err.message))
    }

    const putStudent = (id: number) => {
        fetch(`${FETCH}/${id}`, {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ name: editName, email: editEmail })
        })
            .then(() => { setEditName(""); setEditEmail(""); getStudents() })
            .catch(err => setError(err.message))
    }

    const deleteStudent = (id: number) => {
        fetch(`${FETCH}/${id}`, {
            method: "DELETE"
        })
            .then(() => getStudents())
            .catch(err => setError(err.message))
    }

    if (loading) return <p>Loading...</p>
    if (error) return <p style={{ color: "red" }}>Error: {error}</p>

    return (
        <div>
            <h1>Students</h1>
            {apiData?.map((s: any) => (
                <div key={s.studentId}> <span style={{ textAlign: "left", fontWeight: "bold" }}>{s.name}</span>
                    <p>{s.studentId}, {s.name}, {s.email} </p>

                </div>
            ))}
            <input value={newName} onChange={e => setNewName(e.target.value)} placeholder="Name" />
            <input value={newEmail} onChange={e => setNewEmail(e.target.value)} placeholder="Email" />
            <button onClick={postStudent}>Add Student</button>
            <input value={editName} onChange={e => setEditName(e.target.value)} placeholder="Edit Name" />
            <input value={editEmail} onChange={e => setEditEmail(e.target.value)} placeholder="Edit Email" />
            <button onClick={() => putStudent(s.studentId)}>Update Student</button>
            <input value={deleteId} onChange={e => setDeleteId(e.target.value)} placeholder="Delete Student" />
            <button onClick={() => deleteStudent(s.studentId)}>Delete Student</button>
        </div>
```

GIT:

Using Git is very simple.

Install Git Bash.

Create a Git Repository using a free GitHub account, then copy the link for it and follow these commands in your terminal (at the correct file location ex: C:Users\klaitinen\.vscode\app1):

git init
git remote add origin <paste link>
git branch -M main
git add .
git commit -m “initialize”
git push -origin main

And then you will have a git repository of whichever folder you were on.
Use:

cd ..	to go back a level
dir	to see the files in the current level
cd <file name>	to navigate down

Or just use the open in terminal function from your file explorer to get to the proper file.

```
Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         4/21/2026   1:44 PM            313 invoice.json
-a----         4/21/2026   1:10 PM           1622 jest_sum.js
-a----         4/21/2026   1:10 PM            977 jest_sum.test.js
-a----         4/21/2026   1:43 PM            214 plays.json
-a----         4/23/2026   1:41 PM           5511 statement_v0.js
-a----         4/21/2026   1:10 PM            916 sum.js


PS C:\Users\klaitinen\Downloads\Refactoring> git init
Initialized empty Git repository in C:/Users/klaitinen/Downloads/Refactoring/.git/
PS C:\Users\klaitinen\Downloads\Refactoring> git remote add origin https://github.com/klaitinen67/RefactoringPractice.gi
t
PS C:\Users\klaitinen\Downloads\Refactoring> git branch -M main
PS C:\Users\klaitinen\Downloads\Refactoring> git add .
warning: in the working copy of 'jest_sum.js', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'jest_sum.test.js', LF will be replaced by CRLF the next time Git touches it
PS C:\Users\klaitinen\Downloads\Refactoring> git commit -m "make"
[main (root-commit) 6ec5f7e] make
 6 files changed, 285 insertions(+)
 create mode 100644 invoice.json
 create mode 100644 jest_sum.js
 create mode 100644 jest_sum.test.js
 create mode 100644 plays.json
 create mode 100644 statement_v0.js
 create mode 100644 sum.js
PS C:\Users\klaitinen\Downloads\Refactoring> git push -u origin main
```

Updating the GitHub Repo is as simple as
git add .

git commit -m “message”
git push

And the current version will be added into the repository’s history